iranpaper Downloaded from https://iranpaper.ir
https://www.tarjomano.com ترجمه تخصصی این مقاله مورد تایید ایران پیپر

2023 International Conference on Machine Learning and Applications (ICMLA)

Energy-efficient GPU-intensive Workload Scheduling for Data Centers

Matthew Smith*, Luke Zhao†, Jonathan Cordova*, Xunfei Jiang*, and Mahdi Ebrahimi*
Department of Computer Science, California State University, Northridge *
Department of Computer Science, Brown University †
matthew.smith.227@my.csun.edu, luke_zhao@brown.edu, jonathan.cordova.394@my.csun.edu,
{xunfei.jiang, mahdi.ebrahimi}@csun.edu

***Abstract*—Cooling costs count for a significant part of the total energy consumption in data centers, and previous researchers mainly focused on investigating thermal-ware workload distribution strategies for CPU-intensive workloads. This paper introduces a novel machine learning-based approach that aims at reducing energy consumption through thermal-aware workload distribution to build energy-efficient data centers for GPU-intensive workload. To achieve this goal, the study employs the GpuCloudSim Plus simulator, which effectively models the distribution of GPU-intensive applications under diverse workloads and utilizations. The integration of machine learning models allows for accurate temperature predictions and comprehensive evaluation of the proposed algorithm's performance. We proposed a new workload scheduling algorithm, ThermalAwareGpu, to reduce the energy cost for GPU-intensive workload. We evaluated our algorithm by generating three common patterns of workloads, and saved up to 12.79% of computing cost compared to the baseline algorithms. Our future work includes exploring the estimation of data center cooling energy and conducting in-depth comparisons of different workload balancing algorithms on various compute-intensive workloads.**

***Index Terms*—machine learning, energy-efficient, workload management, GPU, data centers**

I. Introduction

Cloud computing has become much more pervasive as increasing numbers of companies are shifting to energy-efficient cloud providers for computing and storage needs [1]. Datacenters consume about 1% of the world's energy, 40% of which is estimated to be from the cost of cooling server hardware [2]. The increasing demand for power in the data center industry necessitates an urgent exploration into energy efficiency. Various strategies have been proposed to reduce energy consumption for data centers. Dispatching tasks to nodes near the bottom of a rack in homogenous rack mount cluster systems resulted in reduction of cooling costs [3]. An adaptive pool based resource management [4] strategy reduced energy with minimum quality of service loss by setting servers into idle states.

Workload balancing is the process of balancing the workload distribution to computer servers and resources. Most proposed research methods for energy-efficient workload management focus on virtual machine (VM) migration. Dynamic Voltage and Frequency Scaling [5] was adopted to adjust power for computer servers and task resource allocation was performed through VM migration to improve performance and maximize energy savings. A two-phase heuristic bin-packing algorithm was introduced to reduce energy consumption through virtual to physical machine mapping and VM migration to reduce the number of active physical machines [6]. Thermal prediction with machine learning models, when applied to VM migration load balancing algorithms, has been shown to significantly reduce CPU temperature within data centers [7]. These thermal-aware algorithms successfully decreased energy usage but consumed additional resources on virtual machine migration. To reduce the overhead, we seek to create and evaluate thermal-aware workload balancing algorithms at the application level.

We proposed a novel machine learning-based approach to optimize energy-efficient workload scheduling in data centers. Leveraging the GpuCloudSim Plus simulator, we effectively modeled the distribution of GPU-intensive applications under diverse workloads and utilization. By strategically distributing GPU-intensive tasks across servers, we aim to maintain high system performance and efficient resource utilization, while minimizing both computing and cooling costs. We integrated machine learning models that enable precise temperature predictions. We proposed a thermal-aware algorithm for GPU-intensive workload distribution and evaluated the performance and energy consumption of our algorithm using three different types of GPU-intensive workloads. Our algorithm saved up to 12.79% of computing cost and was comparable with a well-performing thermal-aware baseline algorithm.

The rest of this paper is organized as follows: Section II discusses related work; Section III presents the design of the project; Section IV discusses the methodologies used in this project; Section V presents the experimental results; and Section VI concludes the project and discusses future work.

II. Related Works

A. Data Center Simulators

GreenCloud was introduced to simulate the environment for cloud computing data centers [8]. It captures details of the energy consumed by data center components (servers, switches, and links) as well as packet-level communication patterns in realistic setups. It could simulate two-tier, three-tier, and three-tier high-speed data center architectures while utilizing various power management schema (i.e., voltage scaling, frequency scaling, and dynamic shutdown). However,

2023 International Conference on Machine Learning and Applications (ICMLA) | 979-8-3503-4534-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICMLA58977.2023.00263

1946-0759/23/$31.00 ©2023 IEEE
DOI 10.1109/ICMLA58977.2023.00263

Authorized licensed use limited to: Universitaetsbibliothek Regensburg. Downloaded on July 19,2024 at 09:08:02 UTC from IEEE Xplore. Restrictions apply.

Downloaded from https://iranpaper.ir
https://www.tarjomano.com ترجمه تخصصی این مقاله مورد تایید ایران پیپر

the GreenCloud simulator is not supported by the authors and is not publicly available.

CloudSim [9] is a commonly used simulator that aims to provide a generalized and extensible simulation framework for cloud computing by supporting modeling, simulation, and experimentation of emerging cloud computing infrastructures and application services. It supports modeling and simulation of virtualized server hosts, with customizable policies for provisioning host resources to virtual machines and energy-aware computational resources. CloudSim Plus [10] is a highly configurable and extensible cloud computing simulator that can model CPU behavior. The software was created as a fork of CloudSim 3, with the objective of limiting code reuse and providing greater extensibility. To support the modeling and simulation of GPU-enabled VMs in cloud data centers, GPUCloudSim [11] was proposed and was later migrated to an extension for CloudSim Plus, creating GpuCloudSim Plus [12].

B. Thermal-aware Workload Scheduling for Data Centers

Various strategies have been proposed to reduce the computing cost of computer servers in data centers, while the cooling cost has not been fully investigated. Moore et al. developed real-world scheduling algorithms, zone-based discretization (ZBD) and minimize-heat-recirculation (MinHR), by leveraging the non-intuitive observation that the source of cooling inefficiencies can often be in locations spatially uncorrelated with its manifested consequences [13]. GRANITE [14], a virtual machine scheduling algorithm, was proposed to minimize total datacenter energy consumption and evaluated it using real Cloud workload characteristics extracted from Google datacenter trace logs. Their results demonstrated that GRANITE consumed 4.3% to 43.6% less total energy and reduced the probability of critical temperature violation by 99.2%.

Guo et al. proposed a temperature-aware Virtual Data Center (VDC) embedding scheme to avoid hot spots by minimizing the maximum temperature of hot air emitted from each rack, aiming at reducing the total power consumption of IT equipment [15]. They developed a Mixed Integer Linear Programming (MILP) model and a heuristic algorithm to implement the VDC embedding scheme. Wei et al. used a deep reinforcement learning (DPR) to decrease heating, ventilation, and air conditioning (HVAC) energy consumption and maintain room temperature in data centers housed in mixed use buildings [16]. Their methods controlled HVAC systems to reduce costs of operating these cooling systems by 26.5%, with an additional 5.5% saved when using the DRP-based load balancer. Ilager et al. introduced a dynamic scheduling algorithm to minimize the peak temperature of hosts. They claimed that their algorithm reduced the peak temperature by 6.5 °C and consumed 34.5% less energy when compared to the baseline algorithm. Li et al. proposed a thermal-aware workload distribution strategy to improve the energy efficiency of cloud data centers which lowered the cooling cost by 15.4% to 50% [17].

III. Design

The design of the project is shown in Fig. 1. The GpuCloudSim Plus simulator is an integration of GPUCloudSim and CloudSim Plus. The Workload Generator creates batches of workload traces, which are used to generate GpuCloudlets at specified timestamps. A GpuCloudlet includes two parts: (1) a Cloudlet object that represents a CPU task; and (2) a GpuTask object that represents a GPU task. The metrics that are generated to form a GpuCloudlet include: GpuTask and Cloudlet lengths, GPU utilization, GRAM utilization, and CPU utilization. After the creation of the GpuCloudlet instances, the GpuDatacenterBroker (workload balancer) dispatches them to virtual machines for execution.

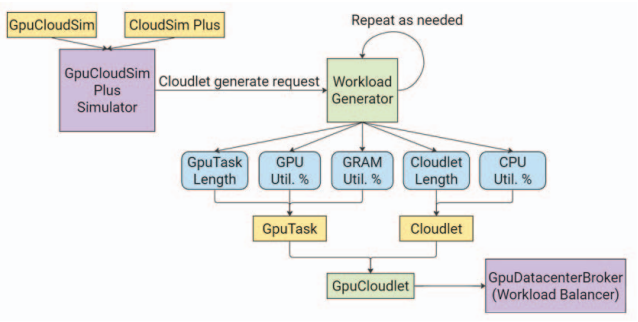


Fig. 1. Project Design

A. Workload Generation

To replicate real-world workloads found in data centers, three common patterns of data traffic were utilized: random, intermittent (bursty), and periodic. These three types of workload used uniform random distributions to generate the CPU utilization, GRAM utilization, GPU utilization, and GpuTask length. Cloudlets per second varied between workload types.

1) Random Workload: This pattern exhibits relatively unchanging utilization, resembling typical steady-state data traffic. Cloudlets per second was uniformly distributed.

2) Periodic Workload: The periodic workload replicates traffic where certain tasks or processes are scheduled to run at fixed intervals or according to a predefined schedule, simulating repetitive and predictable patterns of data traffic. This was modeled using a sine function to generate a smooth, periodic oscillation in workload intensity. It has the form:

$$C(t) = A \cdot \sin(2\pi t/T) + \phi \tag{1}$$

where:

- $C(t)$ represents the Cloudlets generated at time t,
- A is the amplitude of the oscillation,
- T is the period of the oscillation,
- ϕ is the average Cloudlet number

3) Intermittent (Bursty) Workload: The intermittent workload mimics spikes in user requests, periodic batch processing, or fluctuating network traffic, reflecting scenarios where activity experiences sudden bursts followed by periods of low demand. Cloudlets per second were uniformly distributed.

Authorized licensed use limited to: Universitaetsbibliothek Regensburg. Downloaded on July 19,2024 at 09:08:02 UTC from IEEE Xplore. Restrictions apply.

iranpaper Downloaded from https://iranpaper.ir
https://www.tarjomano.com ترجمه تخصصی این مقاله مورد تایید ایران پیپر

During spikes, we applied a spike multiplier constant to the Cloudlet number to simulate bursts in data traffic.

B. Thermal-Aware Workload Schedulers

1) ThermalAwareBaseline: We took inspiration from an algorithm from existing research [7] designed for virtual migration policies on the CPU, and used it as a thermal-aware baseline method for comparison purposes. The original algorithm prioritizes hosts by minimum temperature and verifies that they have resources to support a virtual machine. If a host is over a maximum threshold for either CPU utilization or CPU temperature, the host is skipped for evaluation. Our adapted version of the algorithm checks for GPU utilization and temperature thresholds in a host, prioritizing hosts with lower GPU temperatures. A virtual machine with resources will be selected from the host in accordance with the best fit algorithm.

2) ThermalAwareGpu: In a data center, obtaining server diagnostics for hundreds to thousands of hosts in real-time would have a significant amount of overhead. We designed a thermal-aware algorithm, ThermalAwareGpu, that retrieves the GPU temperatures of each host every 10 seconds to limit the amount of information required by the load balancer (as shown in Algorithm 1). The algorithm was designed to quickly find virtual machines in servers with the lowest GPU temperatures.

Algorithm 1 ThermalAwareGpu

procedure GETVMFORCLOUDLET(CLOUDLET)
  $X = 0.5 \times (numVMs \ / \ numHosts)$
  if $timeInSecsSinceLastSort >= 10$ **then**
    Sort $hostQueue$ by host GPU temperature
  end if

  $count \leftarrow -1$
  $foundVm \leftarrow$ null
  while $(!foundVm)$ && $(++count < numHosts)$ **do**
    $host \leftarrow hostQueue.peek()$
    $foundVm \leftarrow$ First VM with resources in $host$
    if $foundVm \neq$ null **then**
      Dequeue $foundVm$, enqueue to $host$
    else
      Dequeue $host$, enqueue to $hostQueue$
    end if
  end while

  if $foundVm.hostId \neq lastHostId$ **then**
    $lastHostId \leftarrow foundVm.hostId$
    $host.numCloudlets \leftarrow 1$
  else if $++host.numCloudlets >= X$ **then**
    Dequeue $host$, enqueue to $hostQueue$
    $host.numCloudlets \leftarrow 0$
  end if

  return $foundVm$
end procedure

The $hostQueue$ is a queue composed of hosts, with each host is represented as a queue of virtual machines. Every 10 seconds, $hostQueue$ is sorted by host GPU temperature. Each host's internal queue of virtual machines is sorted by their last update time.

The algorithm obtains the first host inside $hostQueue$, and assigns it to $host$. $count$ tracks the number of hosts that have been checked by the algorithm. The $foundVm$ variable is set using a first fit algorithm to check for available resources in virtual machines. If no virtual machine is found from the host, we push $host$ to the tail of the $hostQueue$, and the process repeats until a virtual machine with the required resources is found. If a virtual machine is found, the $foundVm$ will be moved to the tail of $host$ to sort by last update time.

Additional checks are made to ensure that no host receives too many Cloudlets per temperature update interval. $host.numCloudlets$ is used to track the number of Cloudlets that are recently assigned to the current host. When it reaches a threshold of $0.5 \times (numVMs \ / \ numHosts)$, the $host$ is moved to the tail of the $hostQueue$ to evenly distribute the incoming Cloudlets across available servers.

IV. METHODOLOGIES

A. Consistent Simulation Data Generation

As mentioned in Section III, uniform random distributions were used to generate data pertaining to Cloudlets. Before a simulation started, copies of the random seeds were generated for each workload object to ensure that the same workloads would be generated in all simulations.

B. Machine Learning Models for Prediction

Three XGBoost machine learning models were used for computing cost prediction for GPU servers:

1) Model 1: XGBoost CPU Temperature Model [2]: takes time (in seconds) and CPU utilization as inputs.

2) Model 2: XGBoost GPU Power Model: takes GPU temperature, GRAM utilization, GPU utilization, CPU temperature, and CPU utilization as inputs.

3) Model 3: XGBoost GPU Temperature Model: takes GPU power, GRAM utilization, GPU utilization, CPU temperature, and CPU utilization as inputs.

Every second, the simulator obtains server data for each host. Fig. 2 shows the relationship between this simulated data and the inputs for the machine learning models. Server diagnostic data and model output for CPU temperature, GPU power, and GPU temperature are stored for performance evaluation at the end of the simulation. The last predicted GPU temperature is saved to repeat the model prediction process and is used by the thermal aware algorithms as server GPU temperature.

The data used to train the XGBoost GPU models shows GPU power may fluctuate wildly second to second (up to a 33W difference), whereas GPU temperature increases or decreases slowly with no significant spikes. Since the GPU power model requires GPU temperature as an input, using the last predicted GPU temperature as input to the power model

Authorized licensed use limited to: Universitaetsbibliothek Regensburg. Downloaded on July 19,2024 at 09:08:02 UTC from IEEE Xplore. Restrictions apply.

iranpaper Downloaded from https://iranpaper.ir
https://www.tarjomano.com ترجمه تخصصی این مقاله مورد تایید ایران پیپر

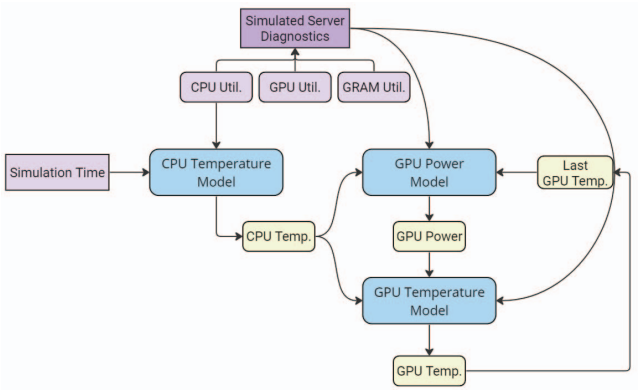


Fig. 2. Machine learning model process

should yield a close approximation of both the current GPU power and temperature inside the GPU.

C. GpuTask and Cloudlet Length Scaling

Both components of a GpuCloudlet (the GpuTask and Cloudlet) have distinct application lengths and utilizations, and they are processed differently by the simulator. Therefore, one component of the simulated GpuCloudlet application can finish before the other is complete. If the application does not represent a real world task, output data obtained from the machine learning models may be inaccurate. To avoid this problem, we scaled the Cloudlet length to match the expected runtime of the GpuTask. We modeled the expected runtime of CPU or GPU tasks as:

$$RT = \frac{L}{V * U} \tag{2}$$

where RT is the expected runtime of a task, L is the length of the task, V is the virtual machine's MIPS capacity, and U is the utilization for a computing resource (CPU or GPU utilization). We can scale L_{cpu} (Cloudlet length) with the knowledge that the expected runtimes need to be equivalent (see Eq. 2):

$$RT_{cpu} = \frac{L_{cpu}}{V_{cpu} * U_{cpu}} = \frac{L_{gpu}}{V_{gpu} * U_{gpu}} = RT_{gpu} \tag{3}$$

$$\implies L_{cpu} = \frac{L_{gpu}}{V_{gpu} * U_{gpu}} \times (V_{cpu} * U_{cpu})$$

Since L_{cpu} is a decimal value and task length is defined as an integer, we round L_{cpu} to its nearest integer value. We rescale the CPU utilization to have an exact match between the GpuTask and Cloudlet runtimes by rewriting Eq. 3:

$$U_{cpu} = \frac{V_{gpu} * U_{gpu}}{L_{gpu}} \times \frac{L_{cpu}}{V_{cpu}}$$

D. Calculating Total Computational Energy

The total computing power P_{comp} was calculated as follows:

$$P_{comp} = \sum_{i=1}^{n} (P_{gpu_i} + P_{cpu_i})$$

where n is the number of hosts and P_{gpu_i}, P_{cpu_i} are the powers drawn from host i by the GPU and CPU, respectively. The total computational energy E_{comp} can be calculated as:

$$E_{comp} = P_{comp} \times ST \tag{4}$$

where ST is simulation runtime.

V. Results

We created a fork[1] of the GPUCloudSim Plus GitHub repository [12] and fixed the compile and runtime errors in the simulator. To evaluate the performance of the ThermalAwareGpu algorithm, we conducted experiments to compare it with the ThermalAwareBaseline algorithm and three built-in algorithms in GPUCloudSim Plus: FirstFit, BestFit, and RoundRobin.

The configuration of the testbed computer server used for this project is shown in Table I. The server has an NVIDIA Tesla P4 GPU and two CPU chips, each with 10 cores. Thirteen different GPU benchmarks, which represent a wide range of real-world applications, were run on the testbed server to train the GPU machine learning models. The parameters for each experiment set were based on the range of training data for the models.

TABLE I
Cluster System Configuration

CPU	2 * Intel(R) Xeon(R) CPU E5-2690 v2 @ 3.00GHz (10 Cores)
Memory	98,304 MB
Disk	2 * 480 GB
GPU	NVIDIA Tesla P4 8GB GDDR5
OS	Ubuntu 22.04.2 LTS

A. Simulation Settings

We simulated a data center of four hosts, with each host having 20 CPU cores and 500 GPU cores. Each host had five virtual machines; each virtual machine had 4 CPU cores (4,000 MIPS capacity) and 100 GPU cores (50,000 MIPS capacity) allocated to them. GpuCloudlets were sent in batches every second by a workload generator. After one simulated hour, GpuCloudlets were no longer submitted to the data center and the simulation was naturally allowed to terminate. Two experiment sets were used to evaluate the algorithms under different conditions for each workload.

B. Experiment Set 1 - Wide Data Range

Experiment set 1 uses the full range of data in the model training set for both GPU and GRAM utilization. The GPU utilization for an application could vary between 25-95% and the GRAM utilization between 5-95%.

GPU temperature and computing energy for this experiment set is shown in Table II. Energy savings were most noticeable in the ThermalAwareBaseline algorithm and the proposed ThermalAwareGpu was within 0.33% energy consumption of the baseline. Compared to the next best performing algorithm, ThermalAwareGpu consumed 9.17% less energy. The performance of ThermalAwareGpu contrasted with other algorithms is shown in Fig. 3.

[1]https://github.com/CSUN-REU/GPUCloudSimPlus

Authorized licensed use limited to: Universitaetsbibliothek Regensburg. Downloaded on July 19,2024 at 09:08:02 UTC from IEEE Xplore. Restrictions apply.

Downloaded from https://iranpaper.ir
https://www.tarjomano.com ترجمه تخصصی این مقاله مورد تایید ایران پیپر

TABLE II
EXPERIMENT SET 1 EVALUATION METRICS

Evaluation Metric	Algorithm Name	Workload Type		
		Random	*Periodic*	*Intermittent*
Computing Energy (KJ)	BestFit	1000.35	990.07	995.63
	FirstFit	1016.67	1007.77	1009.42
	RoundRobin	1017.04	1007.62	1009.36
	TABaseline	904.23	893.94	905.14
	TAGpu	907.08	894.87	910.36
GPU Temp (°C)	BestFit	31.20	31.31	31.16
	FirstFit	31.21	31.36	31.14
	RoundRobin	31.22	31.37	31.15
	TABaseline	31.22	31.23	31.11
	TAGpu	31.26	31.24	31.15

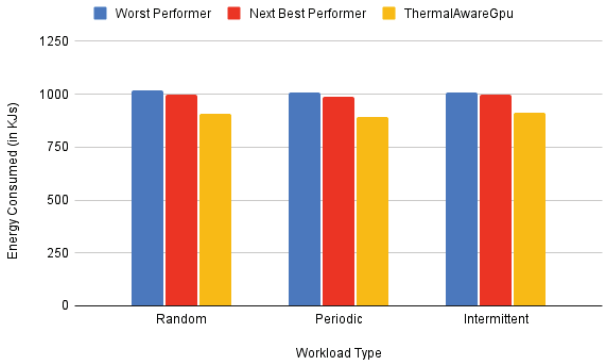


Fig. 3. Energy consumption for algorithms in experiment set 1.

Temperature data varied significantly across all workloads with no clear patterns to identify. The large ranges in GPU and GRAM utilization will lead to the machine learning models being given unexpected input data, such as a near 90% GRAM usage with a 25% GPU utilization, which is not present in our dataset. Additionally, the models did not have much training within the range of 30-60% GPU utilization, which may also explain inconsistent temperature results.

C. Experiment Set 2 - Intense GPU Usage

Experiment set 2 uses a narrower, more intense range of GPU data to simulate more GPU-intensive applications. GPU utilization varied between 75-100% and GRAM utilization was within 50-100%.

TABLE III
EXPERIMENT SET 2 EVALUATION METRICS

Evaluation Metrics	Algorithm Name	Workload Type		
		Random	*Periodic*	*Intermittent*
Computing Energy (KJ)	BestFit	987.27	986.99	980.4
	FirstFit	1008.02	1007.78	999.79
	RoundRobin	1008.02	1007.78	999.79
	TABaseline	890.29	878.06	876.17
	TAGpu	895.06	878.92	877.78
GPU Temp (°C)	BestFit	31.73	31.69	31.65
	FirstFit	31.74	31.72	31.64
	RoundRobin	31.74	31.72	31.64
	TABaseline	31.46	31.46	31.45
	TAGpu	31.47	31.46	31.47

The metrics used to evaluate experiment set 2 are shown in Table III. The ThermalAwareBaseline algorithm consistently performed the best in regards to computational energy savings on all workload types. The ThermalAwareGpu algorithm performed similarly, with an average of 0.27% more energy consumed than the baseline algorithm. ThermalAwareGpu saved on average 10.25% compared to the next best algorithm. Its comparison to other algorithms is shown in Fig. 4.

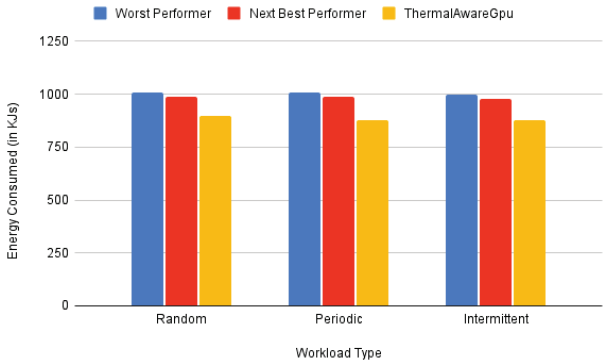


Fig. 4. Energy consumption for algorithms in experiment set 2.

With more intensive GPU applications, there were slight decreases in temperature (less than 1%) in both thermal aware algorithms compared to next best performers. The differences in temperature between the thermal aware algorithms is negligible. The workload traces showing VM utilization in experiment set 2 are shown in Fig. 5, Fig. 6, and Fig. 7.

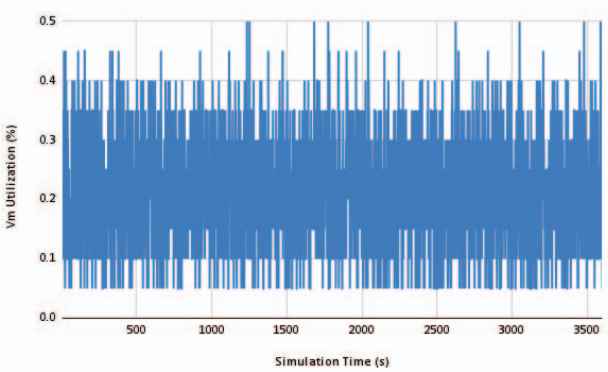


Fig. 5. Random workload data center VM utilization.

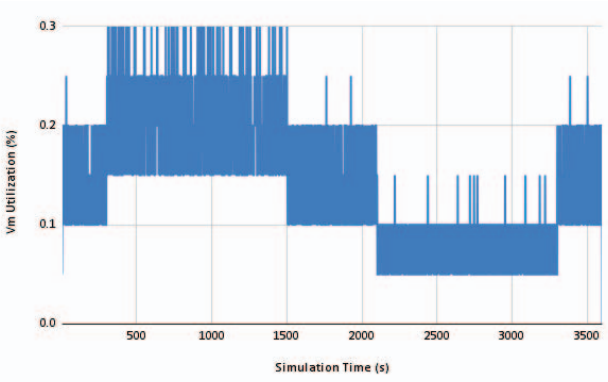


Fig. 6. Periodic workload data center VM utilization.

Authorized licensed use limited to: Universitaetsbibliothek Regensburg. Downloaded on July 19,2024 at 09:08:02 UTC from IEEE Xplore. Restrictions apply.

Downloaded from https://iranpaper.ir
https://www.tarjomano.com ترجمه تخصصی این مقاله مورد تایید ایران پیپر

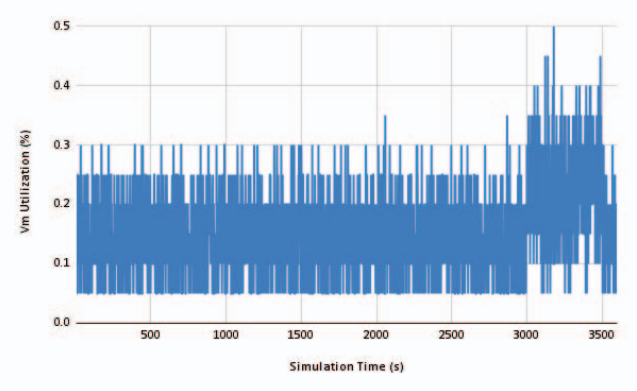


Fig. 7. Intermittent workload data center VM utilization.

Our ThermalAwareGpu algorithm had similar performance as the baseline algorithm and had significant computational energy savings compared to the other evaluated algorithms. The savings were more pronounced and temperature savings became noticeable when using more intense GPU and GRAM utilization distributions.

VI. Conclusion and Future Work

In this project, we investigated the workload scheduling algorithms to reduce the energy consumption of data centers. We developed a workload generator which could create 3 different types of workloads and proposed a ThermalAwareGpu algorithm for GPU-intensive workload scheduling. Our experimental results demonstrated promising energy savings on computing cost, with an average saving of 9.71% for both experiment sets and up to 12.79% savings. Our algorithm performed as well as the ThermalAwareBaseline algorithm, with a maximum of 0.57% more energy consumption. ThermalAwareGpu's allocations would consume less resources than the greedy ThermalAwareBaseline algorithm in large data centers. The 10-second periodic temperature retrieval in our algorithm could reduce overhead and provide additional benefits in a real-world application. Our machine learning-based approach holds the potential to revolutionize energy-efficient workload management in data centers, contributing to sustainable practices and cost-effective operations in this critical domain. As data centers continue to play a pivotal role in technological advancements, our research offers significant promise in enhancing their environmental sustainability and economic efficiency.

In the future, we will work to improve the performance of the machine learning model for power prediction. We will run the simulator over longer time intervals, and evaluate the performance of our proposed workload scheduling algorithm on various combination of workloads. We will also increase the number of hosts and virtual machines and increase the intensity of the simulation to further investigate the performance of our algorithm. Furthermore, we will evaluate our algorithms using real-world data center workload traces.

Acknowledgment

This project is supported by the National Science Foundation under Grant CNS-2244391. We thank Rachel Finley, from Santa Monica College, for building the GPU machine learning models used in this project.

References

[1] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, 2020.

[2] I. Nisce, X. Jiang, and S. P. Vishnu, "Machine learning based thermal prediction for energy-efficient cloud computing," in *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, 2023, pp. 624–627.

[3] Q. Tang, S. K. S. Gupta, and G. Varsamopoulos, "Energy-efficient thermal-aware task scheduling for homogeneous high-performance computing data centers: A cyber-physical approach," *IEEE Transactions on Parallel and Distributed Systems*, vol. 19, no. 11, pp. 1458–1472, 2008.

[4] Z. Xue, X. Dong, S. Ma, S. Fan, and Y. Mei, "An energy-efficient management mechanism for large-scale server clusters," in *The 2nd IEEE Asia-Pacific Service Computing Conference (APSCC 2007)*, 2007, pp. 509–516.

[5] A. Javadpour, A. K. Sangaiah, P. Pinto, F. Ja'fari, W. Zhang, A. M. H. Abadi, and H. Ahmadi, "An energy-optimized embedded load balancing using dvfs computing in cloud data centers," *Computer Communications*, vol. 197, pp. 255–266, 2023.

[6] P. Nehra and N. Kesswani, "Efficient resource allocation and management by using load balanced multi-dimensional bin packing heuristic in cloud data centers," *The Journal of Supercomputing*, vol. 79, no. 2, pp. 1398–1425, 2023.

[7] S. Ilager, K. Ramamohanarao, and R. Buyya, "Thermal prediction for efficient energy management of clouds using machine learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 5, pp. 1044–1056, 2021.

[8] D. Kliazovich, P. Bouvry, Y. Audzevich, and S. U. Khan, "Greencloud: A packet-level simulator of energy-aware cloud computing data centers," in *2010 IEEE Global Telecommunications Conference GLOBECOM 2010*, 2010, pp. 1–5.

[9] R. N. Calheiros, R. Ranjan, C. A. D. Rose, and R. Buyya, "Cloudsim: A novel framework for modeling and simulation of cloud computing infrastructures and services," *arXiv preprint arXiv:0903.2525*, Mar 2009.

[10] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, "Cloudsim plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, 2017, pp. 400–406.

[11] A. Siavashi and M. Momtazpour, "Gpucloudsim: An extension of cloudsim for modeling and simulation of gpus in cloud data centers," *Journal of Supercomputing*, vol. 75, p. 2535–2561, 2019.

[12] M. Hosseini, "Gpucloudsim plus," 2023, last accessed 2 Oct 2023. [Online]. Available: https://github.com/ImMohammadHosseini/GpuCloudsimPlus

[13] J. D. Moore, J. S. Chase, P. Ranganathan, and R. K. Sharma, "Making scheduling "cool": Temperature-aware workload placement in data centers," in *USENIX Annual Technical Conference, General Track*, 04 2005, pp. 61–75.

[14] X. Li, P. Garraghan, X. Jiang, Z. Wu, and J. Xu, "Holistic virtual machine scheduling in cloud datacenters towards minimizing total energy," *IEEE Transactions on parallel and distributed systems*, vol. 29, no. 6, pp. 1317–1331, 2017.

[15] C. Guo, K. Xu, G. Shen, and M. Zukerman, "Temperature-aware virtual data center embedding to avoid hot spots in data centers," *IEEE Transactions on green communications and networking*, vol. 5, no. 1, pp. 497–511, 2020.

[16] T. Wei, S. Ren, and Q. Zhu, "Deep reinforcement learning for joint datacenter and hvac load control in distributed mixed-use buildings," *IEEE Transactions on Sustainable Computing*, vol. 6, no. 3, pp. 370–384, 2021.

[17] J. Li, Y. Deng, Y. Zhou, Z. Zhang, G. Min, and X. Qin, "Towards thermal-aware workload distribution in cloud data centers based on failure models," *IEEE Transactions on Computers*, vol. 72, no. 2, pp. 586–599, 2022.

Authorized licensed use limited to: Universitaetsbibliothek Regensburg. Downloaded on July 19,2024 at 09:08:02 UTC from IEEE Xplore. Restrictions apply.